

Klase mokiniai

Sugeneruota Doxygen 1.13.2

1 Programos versijos	1
1.0.1 Versijos istorija:	1
1.1 Naudojimo instrukcija	1
1.1.1 Failų generavimas:	1
1.1.2 Failų paleidimas:	1
1.1.3 Sugeneruotų failų ištrynimasis:	1
1.1.4 Naudojimo pavyzdžiai:	2
1.2 Spartos analizė	2
1.2.1 Vector	2
1.2.2 List	2
1.2.3 Deque	2
1.2.4 Deque naudojant klasę (be flag)	2
1.2.5 Deque naudojant klasę (-O2)	2
1.2.6 Deque naudojant klasę (-O3)	2
1.2.7 Deque naudojant struct (-O3)	3
1.3 Sistemos parametrai	3
1.4 Išvados	3
1.5 Perdengtų metodų paaiškinimas	3
2 Hierarchijos Indeksas	5
2.1 Klasių hierarchija	5
3 Klasės Indeksas	7
3.1 Klasės	7
4 Failo Indeksas	9
4.1 Failai	9
5 Klasės Dokumentacija	11
5.1 Stud Struktūra	11
5.1.1 Smulkus aprašymas	13
5.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	13
5.1.2.1 Stud() [1/4]	13
5.1.2.2 Stud() [2/4]	13
5.1.2.3 ~Stud()	14
5.1.2.4 Stud() [3/4]	14
5.1.2.5 Stud() [4/4]	15
5.1.3 Metodų Dokumentacija	16
5.1.3.1 getEgzaminas()	16
5.1.3.2 getMediana()	16
5.1.3.3 getPazymiai()	17
5.1.3.4 getVidurkis()	17
5.1.3.5 operator=() [1/2]	17
5.1.3.6 operator=() [2/2]	17

5.1.3.7 setEgzaminas()	18
5.1.3.8 setMediana()	18
5.1.3.9 setPazymiai()	19
5.1.3.10 setVidurkis()	19
5.1.3.11 Skaityti()	19
5.1.4 Atributų Dokumentacija	20
5.1.4.1 egz	20
5.1.4.2 med	20
5.1.4.3 pav	21
5.1.4.4 paz	21
5.1.4.5 vard	21
5.1.4.6 vid	21
5.2 Zmogus Klasė	21
5.2.1 Smulkus aprašymas	22
5.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	22
5.2.2.1 Zmogus() [1/2]	22
5.2.2.2 Zmogus() [2/2]	22
5.2.2.3 ~Zmogus()	23
5.2.3 Metodų Dokumentacija	23
5.2.3.1 getPavarde()	23
5.2.3.2 getVardas()	23
5.2.3.3 setPavarde()	23
5.2.3.4 setVardas()	24
5.2.3.5 Skaityti()	24
5.2.4 Atributų Dokumentacija	25
5.2.4.1 pav	25
5.2.4.2 vard	25
6 Failo Dokumentacija	27
6.1 deque/deque.cpp Failo Nuoroda	27
6.1.1 Funkcijos Dokumentacija	27
6.1.1.1 main()	27
6.2 deque/functions2.cpp Failo Nuoroda	28
6.2.1 Funkcijos Dokumentacija	29
6.2.1.1 Auto()	29
6.2.1.2 Ekrane()	30
6.2.1.3 Faile()	31
6.2.1.4 GeneruotiFaila()	32
6.2.1.5 Manual()	32
6.2.1.6 Rusiuoti()	33
6.2.1.7 Semi()	34
6.2.1.8 Skirstymas1()	35

6.2.1.9 Skirstymas2()	35
6.2.1.10 Skirstymas3()	36
6.2.1.11 testRuleOfFive()	37
6.3 deque/header2.h Failo Nuoroda	37
6.3.1 Funkcijos Dokumentacija	39
6.3.1.1 Auto()	39
6.3.1.2 Ekrane()	40
6.3.1.3 Faile()	40
6.3.1.4 GeneruotiFaila()	41
6.3.1.5 Manual()	41
6.3.1.6 Rusiuoti()	42
6.3.1.7 Semi()	43
6.3.1.8 Skirstymas1()	44
6.3.1.9 Skirstymas2()	44
6.3.1.10 Skirstymas3()	45
6.3.1.11 testRuleOfFive()	46
6.4 header2.h	46
6.5 list/functions1.cpp Failo Nuoroda	48
6.5.1 Funkcijos Dokumentacija	49
6.5.1.1 Auto()	49
6.5.1.2 Ekrane()	49
6.5.1.3 Faile()	49
6.5.1.4 GeneruotiFaila()	50
6.5.1.5 Manual()	50
6.5.1.6 Rusiuoti()	50
6.5.1.7 Semi()	51
6.5.1.8 Skaityti()	51
6.5.1.9 Skirstymas1()	51
6.5.1.10 Skirstymas2()	52
6.5.1.11 Skirstymas3()	52
6.6 list/header1.h Failo Nuoroda	53
6.6.1 Funkcijos Dokumentacija	54
6.6.1.1 Auto()	54
6.6.1.2 Ekrane()	55
6.6.1.3 Faile()	55
6.6.1.4 GeneruotiFaila()	56
6.6.1.5 Manual()	57
6.6.1.6 Rusiuoti()	57
6.6.1.7 Semi()	57
6.6.1.8 Skaityti()	57
6.6.1.9 Skirstymas1()	58
6.6.1.10 Skirstymas2()	58

6.6.1.11 Skirstymas3()	59
6.7 header1.h	60
6.8 list/list.cpp Failo Nuoroda	61
6.8.1 Funkcijos Dokumentacija	61
6.8.1.1 main()	61
6.9 README.md Failo Nuoroda	62
6.10 vector/functions.cpp Failo Nuoroda	62
6.10.1 Funkcijos Dokumentacija	63
6.10.1.1 Auto()	63
6.10.1.2 Ekrane()	63
6.10.1.3 Faile()	63
6.10.1.4 GeneruotiFaila()	64
6.10.1.5 Manual()	64
6.10.1.6 Rusiuoti()	64
6.10.1.7 Semi()	65
6.10.1.8 Skaityti()	65
6.10.1.9 Skirstymas1()	65
6.10.1.10 Skirstymas2()	66
6.10.1.11 Skirstymas3()	66
6.11 vector/header.h Failo Nuoroda	67
6.11.1 Funkcijos Dokumentacija	68
6.11.1.1 Auto()	68
6.11.1.2 Ekrane()	68
6.11.1.3 Faile()	69
6.11.1.4 GeneruotiFaila()	69
6.11.1.5 Manual()	70
6.11.1.6 Rusiuoti()	70
6.11.1.7 Semi()	70
6.11.1.8 Skaityti()	70
6.11.1.9 Skirstymas1()	71
6.11.1.10 Skirstymas2()	71
6.11.1.11 skirstymas3()	72
6.12 header.h	72
6.13 vector/vektorius.cpp Failo Nuoroda	73
6.13.1 Funkcijos Dokumentacija	73
6.13.1.1 main()	73

skyrius 1

Programos versijos

1.0.1 Versijos istorija:

1. **v.pradine** - Pradinė versija. Duomenų įvedimas ranka, surūšiuota išvestis ekrane.
 2. **v0.1** - Studentų ir pažymių generavimas, galimybė pasirinkti tarp vektoriaus ir masyvo naudojimo.
 3. **v0.2** - Failo skaitymas, išvestis į failą, rūšiavimas pagal naudotojo pasirinkimą.
 4. **v0.3** - Geresnė programavimo praktika, pridėtas klaidų apdorojimas (exception handling).
 5. **v0.4** - Studentų skirstymas į pogrupius, failų generavimas.
 6. **v1.0** - Pilna versija. Sukurtas Makefile, kelios versijos su skirtingais konteineriais, optimizacija.
 7. **v1.1** - Pertvarkyta struktūrą pakeičiant į klasę.
 8. **v1.2** - Panaudotas rule of five, implementuoti jo testai.
 9. **v1.2** - Klasė paskirstyta į abstrakčią ir derived klases.
-

1.1 Naudojimo instrukcija

Viskas vykdoma terminale.

1.1.1 Failų generavimas:

```
make all
```

1.1.2 Failų paleidimas:

Pereikite į vykdomųjų failų direktoriją:

```
cd bin
```

- **Vektoriai:** `./vector_program`
- **List:** `./list_program`
- **Deque:** `./deque_program`

1.1.3 Sugeneruotų failų ištrynimasis:

Grįžkite į pradinę direktoriją:

```
cd ..
```

Valymo komanda:

```
make clean
```

1.1.4 Naudojimo pavyzdžiai:

1.2 Spartos analizė

1.2.1 Vector

Studentų skaičius	Failo kūrimas	Failo skaity- mas	Rūšiavima	Paskirs- tymas	Spaus- dinimas	Iš viso	Paskirs- tymas (2)	Paskirs- tymas (3)
1k	0.035 s	0.015 s	0 s	0 s	0.011 s	0.061 s	0.028 s	0 s
10k	0.219 s	0.105 s	0.017 s	0.004 s	0.077 s	0.422 s	0.076 s	0.001 s
100k	1.798 s	0.392 s	0.138 s	0.021 s	0.771 s	3.120 s	180.122 s	0.012 s
1m	16.223 s	7.795 s	0.996 s	0.098 s	9.040 s	34.152 s	>5 min	0.048 s
10m	167.577 s	87.541	12.792	crash	88.551	356.461	crash	crash

1.2.2 List

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirs- tymas	Spausdi- nimas	Iš viso	Paskirs- tymas (2)	Paskirs- tymas (3)
1k	0.012 s	0 s	0 s	0.026 s	0.039 s	0 s	0 s
10k	0.117 s	0.013 s	0.006 s	0.107 s	0.244 s	0 s	0.001 s
100k	0.422 s	0.114 s	0.025 s	0.923 s	1.483 s	0.024 s	0.015 s
1m	7.131 s	0.579 s	0.216 s	9.120 s	17.046 s	0.182 s	0.397 s
10m	86.119 s	10.844 s	crash	89.815 s	188.504 s	1.726 s	4.288 s

1.2.3 Deque

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirs- tymas	Spausdi- nimas	Iš viso	Paskirs- tymas (2)	Paskirs- tymas (3)
1k	0.015 s	0 s	0.002 s	0.029 s	0.046 s	0 s	0 s
10k	0.108 s	0.014 s	0.008 s	0.097 s	0.227 s	0 s	0.003 s
100k	0.407 s	0.155 s	0.029 s	0.919 s	1.510 s	0.025 s	0.012 s
1m	7.214 s	1.092 s	0.102 s	9.293 s	17.701 s	0.059 s	0.059 s
10m	85.353 s	12.767 s	crash	89.707 s	188.283 s	0.456 s	0.470 s

1.2.4 Deque naudojant klasę (be flag)

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirstymas (3)	Spausdinimas	Iš viso
100k	1.231 s	0.367 s	0.031 s	0.944 s	2.574 s
1m	9.717 s	4.968 s	0.251 s	9.448 s	24.384 s

.exe failo dydis - 372 KB

1.2.5 Deque naudojant klasę (-O2)

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirstymas (3)	Spausdinimas	Iš viso
100k	0.954 s	0.100 s	0.007 s	0.892 s	1.954 s
1m	7.641 s	1.027 s	0.042 s	9.015 s	17.725 s

.exe failo dydis - 195 KB

1.2.6 Deque naudojant klasę (-O3)

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirstymas (3)	Spausdinimas	Iš viso
100k	0.820 s	0.069 s	0.007 s	0.717 s	1.614 s
1m	6.125 s	0.875 s	0.051 s	7.403 s	14.455 s

.exe failo dydis - 190 KB

1.2.7 Deque naudojant struct (-03)

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirstymas (3)	Spausdinimas	Iš viso
100k	0.709 s	0.155 s	0.012 s	0.919 s	1.493 s
1m	7.214 s	1.092 s	0.059 s	8.895 s	17.260 s

1.3 Sistemos parametrai

- **CPU:** i7-13650HX
- **RAM:** 24GB 4800MHz
- **Storage:** NVMe M.2 SSD 1TB

1.4 Išvados

Vektoriai veikė prasčiausiai atminties atžvilgiu, crashino. **1 strategija** pasižymi prastu atminties išnaudojimu (su dideliais kiekiais taip pat sukėlė crash). **2 strategija** pasižymi didžiausiu spartumu (išskyrus su vektoriais). Naudojamos atminties kiekis tarp **deque ir list konteinerių** kito minimaliai, todėl spartos atžvilgiu **deque konteineris su strategija Nr. 2** yra geriausias pasirinkimas. Taip pat pastebime, kad **klasė** yra iki 20% spartesnė nei **struktūra**.

Klasės konteineris užtrunka šiek tiek ilgiau skaitymo funkcijoje, tačiau visas kitas funkcijas atlieka greičiau (skirtumas nėra didelis).

Naudojant -O2 ir -O3 **Optimizavimo flag'us** matomas didelis spartos skirtumas palyginus be flag'o. Skirtumas tarp -O2 ir -O3 yra minimalus, bet vistiek pastebime, kad -O3 veikia sparčiausiai. Su -O1 flag'u kilo problemų, yra errorų tarp naudojamos mingw32 versijos ir -O1 flag.

1.5 Perdengtų metodų paaiškinimas

Manual įvestis - sukuriami laikina klasė, panaudojamas default konstruktorius. Vartotojas veda studentų vardus/pavardes arba "stop", kad užbaigtų įvedimą (nėra case sensitive). Vienu metu įvedami visi pažymiai atskirti tarpu, kurie yra saugojami vektoriuje. Įvedamas egzamino pažymys. Iš karto paskaičiuojami mediana ir vidurkiai, kad nereikėtų saugoti visų pažymių, kad sutaupyti atminties. Panaudojamas move konstruktorius perkelti iš laikinos klasės į naudojamą visoje programoje. Destruktorius sunaikina laikiną klasę.

Semi įvestis - sukuriami laikina klasė, panaudojamas default konstruktorius. Vartotojas veda studentų vardus/pavardes arba "stop", kad užbaigtų įvedimą (nėra case sensitive). Randomly sugeneruojami iki 10 pažymių. Iš karto paskaičiuojami mediana ir vidurkiai, kad nereikėtų saugoti visų pažymių, kad sutaupyti atminties. Panaudojamas move konstruktorius perkelti iš laikinos klasės į naudojamą visoje programoje. Destruktorius sunaikina laikiną klasę.

Auto įvestis - sukuriami laikina klasė, panaudojamas default konstruktorius. Sugeneruojami iki 10 studentų (vardai, pavardės parenkami atsitiktinai iš vektorių), kiekvienas iki 10 pažymių. Iš karto paskaičiuojami mediana ir vidurkiai, kad nereikėtų saugoti visų pažymių, kad sutaupyti atminties. Panaudojamas move konstruktorius perkelti iš laikinos klasės į naudojamą visoje programoje. Destruktorius sunaikina laikiną klasę.

Ekrane išvestis - ekrane atspausdina visus surūšiuotus studentus (pagal vardą/pavardę/galutinį balą) ir jų galutinį balą vidurkį/medianą pagal vartotojo pasirinkimą. Rule of five nenaudojamas, kadangi nekuriama nauja klasė (nereikia default konstruktoriaus), neperkeliami ir nekopijuojami duomenys (nereikia move/copy konstruktorius/operatoriaus) ir nenaikinami laikini duomenys (nereikia destruktoriaus).

Failė išvestis - vartotojas pasirenka ar paskirstyti studentus į 2 atskirus failus pagal vidurkį. Skirstant į grupes sukuriami 2 nauji vektoriai, į kuriuos perkeliama duomenys naudojant move konstruktorius, priklausant nuo vartotojo pasirinktos rūšiavimo strategijos. Duomenys atspausdinami failė/failuose pagal vartotojo pasirinktą rūšiavimą. Panaudojamas destruktoriaus, kadangi vektoriams priklausė klasės nariai.

Kilus klausimams ar pastaboms, susisiekiame el. paštu: **nojus .petrusis@mif.stud.vu.lt**

skyrius 2

Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Zmogus	21
Stud	11

skyrius 3

Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

[Stud](#)

Išvestinė klasė 11

[Zmogus](#)

Bazinė/abstrakti klasė 21

skyrius 4

Failo Indeksas

4.1 Failai

Visų failų sąrašas su trumpais aprašymais:

deque/deque.cpp	27
deque/functions2.cpp	28
deque/header2.h	37
list/functions1.cpp	48
list/header1.h	53
list/list.cpp	61
vector/functions.cpp	62
vector/header.h	67
vector/vektorius.cpp	73

skyrius 5

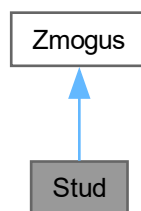
Klasės Dokumentacija

5.1 Stud Struktūra

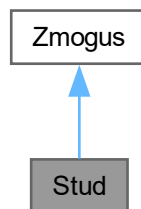
Išvestinė klasė.

```
#include <header2.h>
```

Paveldimumo diagrama Stud:



Bendradarbiavimo diagrama Stud:



Vieši Metodai

- `Stud ()`
Numatytasis konstruktorius.
- `Stud (const string &vardas, const string &pavarde, const vector< int > &pazymiai, int egzaminas)`

- *Konstruktorius su parametrais.*
- `~Stud ()`
- *Destruktorius.*
- `Stud (const Stud &other)`
- *Copy konstruktorius.*
- `Stud & operator= (const Stud &other)`
- *Copy priskyrimo operatorius.*
- `Stud (Stud &&other) noexcept`
- *Move konstruktorius.*
- `Stud & operator= (Stud &&other) noexcept`
- *Move priskyrimo operatorius.*
- `vector< int > getPazymiai () const`
- *Getteriai.*
- `int getEgzaminas () const`
- `double getVidurkis () const`
- `double getMediana () const`
- `void setPazymiai (const vector< int > &pazymiai)`
- *Setteriai.*
- `void setEgzaminas (int egzaminas)`
- `void setVidurkis (double vidurkis)`
- `void setMediana (double mediana)`
- `void Skaityti (deque< Zmogus * > &grupe, double &TotalTime) override`
- *Perrašoma grynoji virtuali funkcija.*

Vieši Metodai inherited from `Zmogus`

- `Zmogus ()`
- *Default konstruktorius.*
- `Zmogus (const string &vardas, const string &pavarde)`
- *Konstruktorius su parametrais.*
- `virtual ~Zmogus ()=default`
- *Virtualus destruktoriaus.*
- `string getVardas () const`
- *Getteriai.*
- `string getPavarde () const`
- `void setVardas (const string &vardas)`
- *Setteriai.*
- `void setPavarde (const string &pavarde)`

Vieši Atributai

- string `vard`
- string `pav`

Privatūs Atributai

- vector< int > `paz`
- int `egz`
- double `vid`
- double `med`

Additional Inherited Members

Apsaugoti Atributai inherited from Zmogus

- string `vard`
- string `pav`

5.1.1 Smulkus aprašymas

Išvestinė klasė.

Struktūra, aprašanti studento duomenis.

5.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.1.2.1 Stud() [1/4]

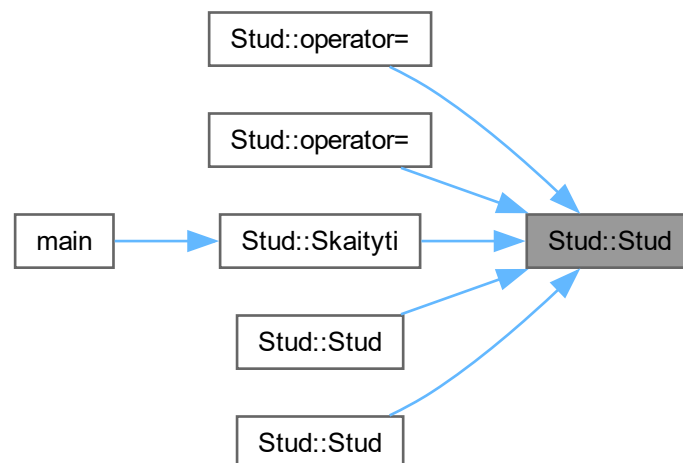
```
Stud::Stud () [inline]
```

Numatytasis konstruktorius.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



5.1.2.2 Stud() [2/4]

```
Stud::Stud (
    const string & vardas,
```

```
const string & pavarde,
const vector< int > & pazymiai,
int egzaminas) [inline]
```

Konstruktorius su parametrais.

Parametrai

<i>vardas</i>	Vardas.
<i>pavarde</i>	Pavardė.
<i>pazymiai</i>	Pažymiai.
<i>egzaminas</i>	Egzamino rezultatas.

Funkcijos kvietimo grafas:



5.1.2.3 ~Stud()

Stud::~~Stud ()
Destruktorius.

5.1.2.4 Stud() [3/4]

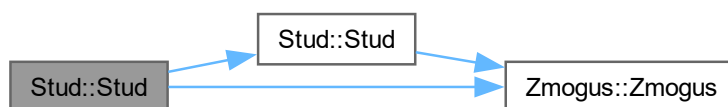
```
Stud::Stud (
    const Stud & other) [inline]
```

Copy konstruktorius.

Parametrai

<i>other</i>	Kitas objektas.
--------------	-----------------

Funkcijos kvietimo grafas:



5.1.2.5 Stud() [4/4]

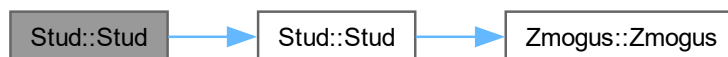
```
Stud::Stud (  
    Stud && other)    [inline], [noexcept]
```

Move konstruktorius.

Parametrai

<i>other</i>	Kitas objektas.
--------------	-----------------

Funkcijos kvietimo grafas:

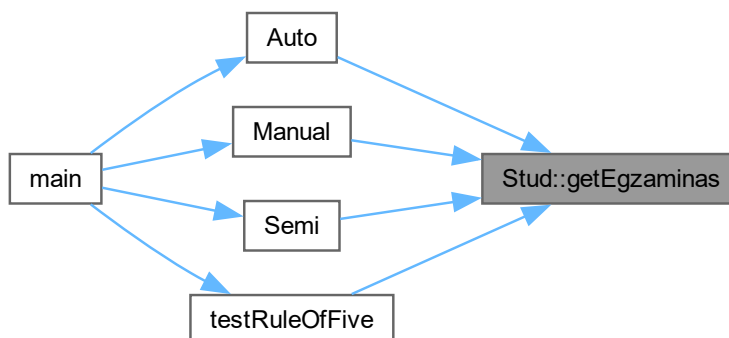


5.1.3 Metodų Dokumentacija

5.1.3.1 getEgzaminas()

```
int Stud::getEgzaminas () const [inline]
```

Here is the caller graph for this function:



5.1.3.2 getMediana()

```
double Stud::getMediana () const [inline]
```

Here is the caller graph for this function:



5.1.3.3 getPazymiai()

```
vector< int > Stud::getPazymiai () const [inline]  
Getteriai.
```

Here is the caller graph for this function:



5.1.3.4 getVidurkis()

```
double Stud::getVidurkis () const [inline]
```

5.1.3.5 operator=() [1/2]

```
Stud & Stud::operator= (  
    const Stud & other) [inline]
```

Copy priskyrimo operatorius.

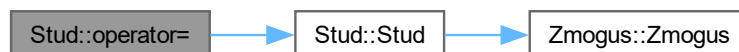
Parametrai

<i>other</i>	Kitas objektas.
--------------	-----------------

Gražina

`Stud&`

Funkcijos kvietimo grafas:



5.1.3.6 operator=() [2/2]

```
Stud & Stud::operator= (  
    Stud && other) [inline], [noexcept]
```

Move priskyrimo operatorius.

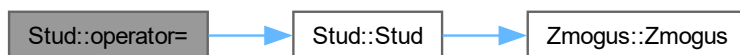
Parametrai

<i>other</i>	Kitas objektas.
--------------	-----------------

Gražina

[Stud&](#)

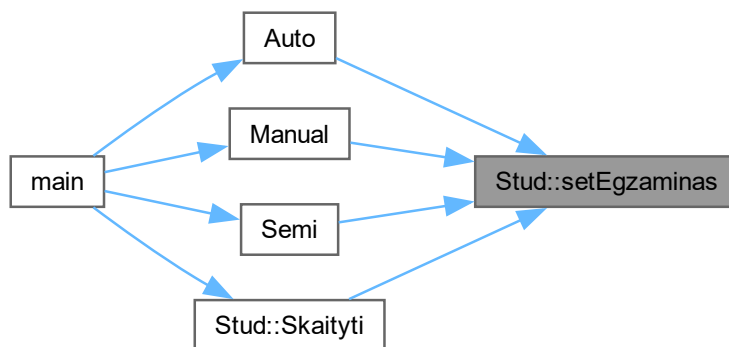
Funkcijos kvietimo grafas:



5.1.3.7 setEgzaminas()

```
void Stud::setEgzaminas (  
    int egzaminas) [inline]
```

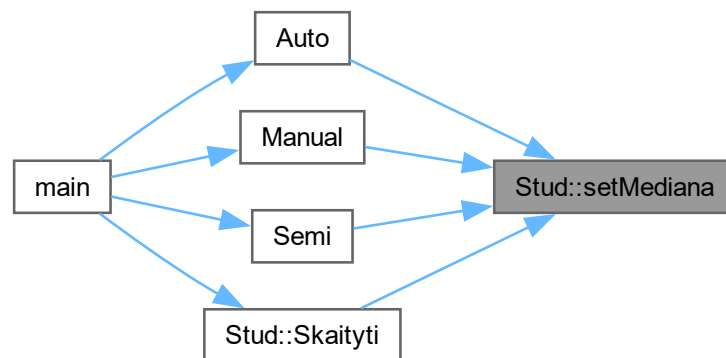
Here is the caller graph for this function:



5.1.3.8 setMediana()

```
void Stud::setMediana (  
    double mediana) [inline]
```

Here is the caller graph for this function:



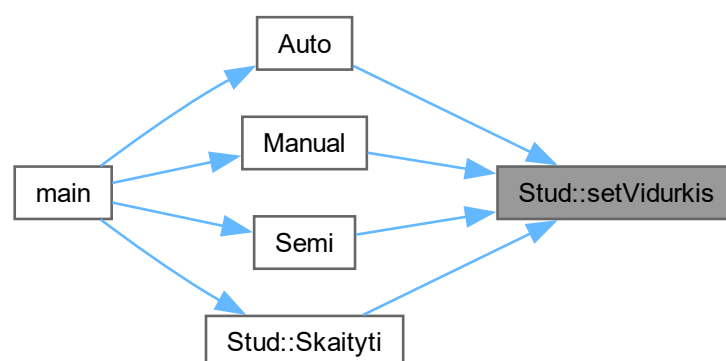
5.1.3.9 setPazymiai()

```
void Stud::setPazymiai (
    const vector< int > & pazymiai) [inline]
Setteriai.
```

5.1.3.10 setVidurkis()

```
void Stud::setVidurkis (
    double vidurkis) [inline]
```

Here is the caller graph for this function:



5.1.3.11 Skaityti()

```
void Stud::Skaityti (
    deque< Zmogus * > & grupe,
```

```
double & TotalTime) [override], [virtual]
```

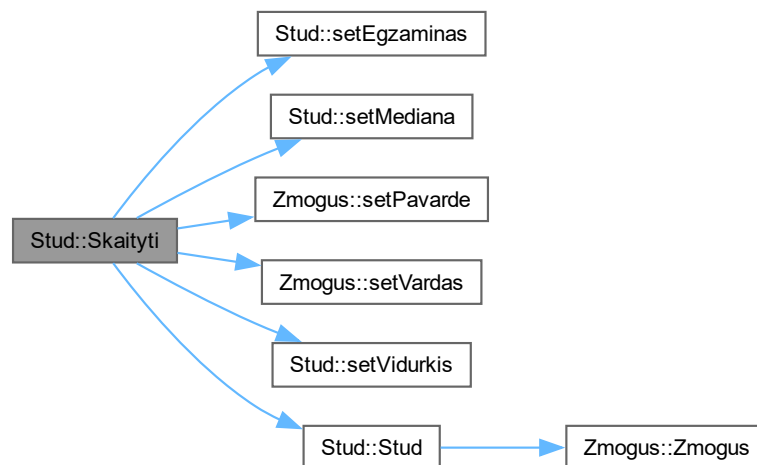
Perrašoma grynoji virtuali funkcija.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>TotalTime</i>	Bendras operacijos laikas.

Realizuoja [Zmogus](#).

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



5.1.4 Atributų Dokumentacija

5.1.4.1 egz

```
int Stud::egz [private]
```

Egzamino rezultatas.

5.1.4.2 med

```
double Stud::med [private]
```

Galutinis rezultatas pagal medianą.

5.1.4.3 pav

```
string Stud::pav
```

Studentų pavardė.

5.1.4.4 paz

```
vector< int > Stud::paz [private]
```

Studentų pažymiai.

5.1.4.5 vard

```
string Stud::vard
```

Studentų vardas.

5.1.4.6 vid

```
double Stud::vid [private]
```

Galutinis rezultatas pagal vidurkį.
Dokumentacija šiai struktūrai sugeneruota iš šių failų:

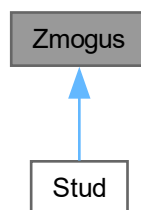
- [deque/header2.h](#)
- [list/header1.h](#)
- [vector/header.h](#)
- [deque/functions2.cpp](#)

5.2 Zmogus Klasė

Bazinė/abstrakti klasė.

```
#include <header2.h>
```

Paveldimumo diagrama Zmogus:

**Vieši Metodai**

- [Zmogus \(\)](#)
Default konstruktorius.
- [Zmogus \(const string &vardas, const string &pavarde\)](#)
Konstruktorius su parametrais.
- `virtual ~Zmogus ()=default`
Virtualus destruktorius.
- `string getVardas () const`
Getteriai.

- string `getPavarde` () const
 - void `setVardas` (const string &vardas)
- Setteriai.*
- void `setPavarde` (const string &pavarde)
 - virtual void `Skaityti` (deque< `Zmogus` * > &grupe, double &TotalTime)=0
- Grynoji virtuali funkcija, kurią reikia įgyvendinti išvestinėse klasėse.*

Apsaugoti Atributai

- string `vard`
- string `pav`

5.2.1 Smulkus aprašymas

Bazinė/abstrakti klasė.

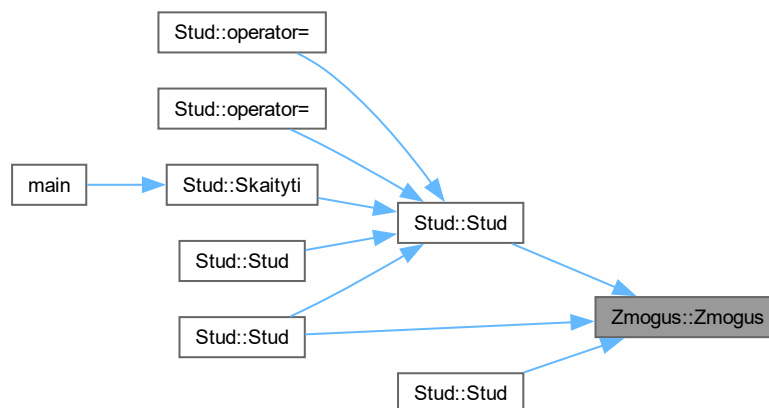
5.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.2.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [inline]
```

Default konstruktorius.

Here is the caller graph for this function:



5.2.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
    const string & vardas,
    const string & pavarde) [inline]
```

Konstruktorius su parametrais.

Parametrai

<code>vardas</code>	Vardas.
<code>pavarde</code>	Pavardė.

5.2.2.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus () [virtual], [default]
```

Virtualus destruktoriaus.

5.2.3 Metody Dokumentacija

5.2.3.1 getPavarde()

```
string Zmogus::getPavarde () const [inline]
```

Here is the caller graph for this function:



5.2.3.2 getVardas()

```
string Zmogus::getVardas () const [inline]
```

Getteriai.

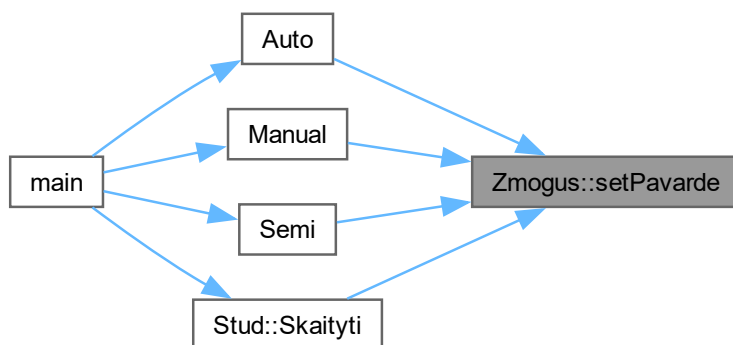
Here is the caller graph for this function:



5.2.3.3 setPavarde()

```
void Zmogus::setPavarde (  
    const string & pavarde) [inline]
```

Here is the caller graph for this function:

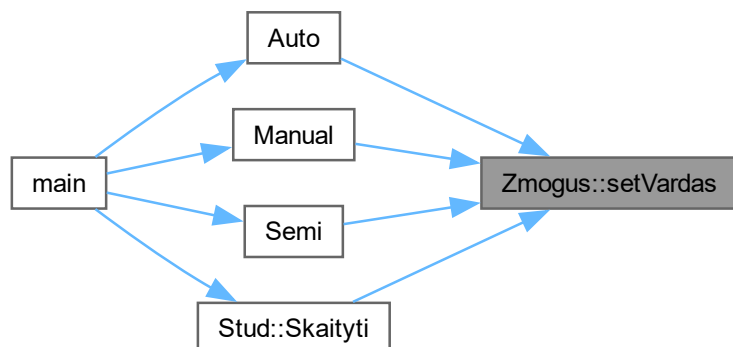


5.2.3.4 setVardas()

```
void Zmogus::setVardas (
    const string & vardas) [inline]
```

Setteriai.

Here is the caller graph for this function:



5.2.3.5 Skaityti()

```
virtual void Zmogus::Skaityti (
    deque< Zmogus * > & grupe,
    double & TotalTime) [pure virtual]
```

Grynoji virtuali funkcija, kurią reikia įgyvendinti išvestinėse klasėse.

Parametrai

<i>grupe</i>	Studentų grupė.
--------------	-----------------

Parametrai

<i>TotalTime</i>	Bendras operacijos laikas.
------------------	----------------------------

Realizuota [Stud.](#)

5.2.4 Atributų Dokumentacija

5.2.4.1 pav

```
string Zmogus::pav [protected]
```

5.2.4.2 vard

```
string Zmogus::vard [protected]
```

Dokumentacija šiai klasei sugeneruota iš šio failo:

- deque/[header2.h](#)

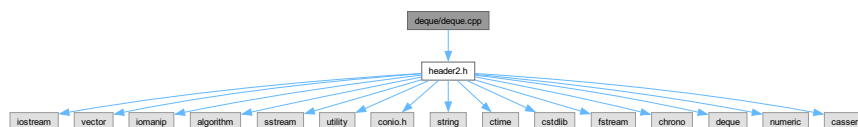
skyrius 6

Failo Dokumentacija

6.1 deque/deque.cpp Failo Nuoroda

```
#include "header2.h"
```

Įtraukimo priklausomybių diagrama deque.cpp:



Funkcijos

- int [main](#) ()

Pagrindinė programos funkcija.

6.1.1 Funkcijos Dokumentacija

6.1.1.1 main()

```
int main ()
```

Pagrindinė programos funkcija.

Ši funkcija leidžia vartotojui pasirinkti įvairias operacijas, tokias kaip studentų duomenų generavimas, įvedimas, rikiavimas, skirstymas ir išvedimas į ekraną arba failą.

Gražina

int Programos vykdymo rezultatas.

Tikrinama, ar vartotojas nori testuoti Rule of Five taisyklę.

Tikrinama, ar vartotojas nori sugeneruoti studentų failą.

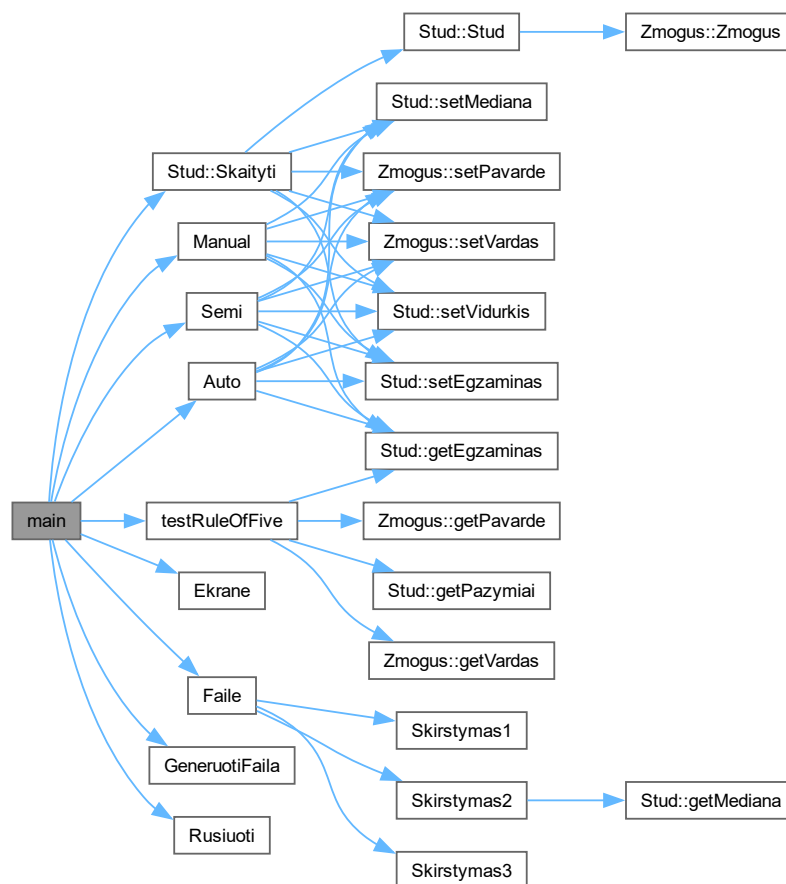
Leidžiama pasirinkti studentų duomenų įvedimo būdą.

Pasirenkama, ar naudoti vidurkį, ar medianą.

Pasirenkamas rikiavimo kriterijus.

Pasirenkama, ar duomenis išvesti į ekraną, ar į failą.

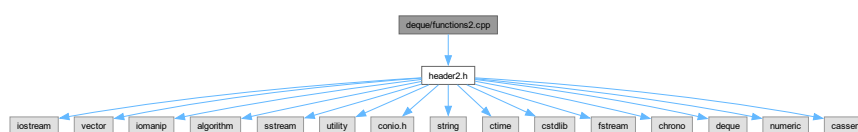
Išvedamas bendras programos veikimo laikas. Funkcijos kvietimo grafas:



6.2 deque/functions2.cpp Failo Nuoroda

```
#include "header2.h"
```

Įtraukimo priklausomybių diagrama functions2.cpp:



Funkcijos

- void **Manual** (**Stud** &laik, deque< **Zmogus** * > &grupe)
Funkcija studentų duomenų įvedimui rankiniu būdu.
- void **Semi** (**Stud** &laik, deque< **Zmogus** * > &grupe)
Funkcija studentų duomenų įvedimui pusiau automatinio būdu.
- void **Auto** (deque< **Zmogus** * > &grupe)
Funkcija studentų duomenų generavimui automatiškai.
- void **Ekrane** (deque< **Zmogus** * > &grupe, char gal, double &TotalTime)
Funkcija studentų duomenų išvedimui į ekraną.
- void **Skirstymas1** (deque< **Zmogus** * > &grupe, char gal, vector< **Stud** > &vargsiukai, vector< **Stud** > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.
- void **Skirstymas2** (deque< **Zmogus** * > &grupe, char gal, vector< **Stud** > &vargsiukai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.
- void **Skirstymas3** (deque< **Zmogus** * > &grupe, char gal, vector< **Stud** > &vargsiukai, vector< **Stud** > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes naudojant `std::partition`.
- void **Failė** (deque< **Zmogus** * > &grupe, char gal, double &TotalTime)
Funkcija studentų duomenų išvedimui į failą.
- void **Rusiuoti** (deque< **Zmogus** * > &grupe, char rusiavimas, char gal, double &TotalTime)
Funkcija studentų rikiavimui pagal pasirinktą kriterijų.
- void **GeneruotiFailą** (double &TotalTime)
Funkcija studentų failo generavimui.
- void **testRuleOfFive** ()
Funkcija Rule of Five taisyklės testavimui.

6.2.1 Funkcijos Dokumentacija

6.2.1.1 Auto()

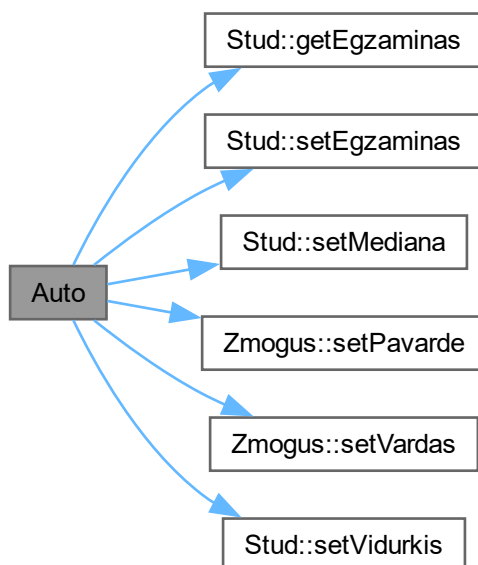
```
void Auto (
    deque< Zmogus * > & grupe)
```

Funkcija studentų duomenų generavimui automatiškai.

Parametrai

<i>grupe</i>	Studentų grupė.
--------------	-----------------

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.2.1.2 Ekrane()

```

void Ekrane (
    deque< Zmogus * > & grupe,
    char gal,
    double & TotalTime)
  
```

Funkcija studentų duomenų išvedimui į ekraną.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.2.1.3 Faile()

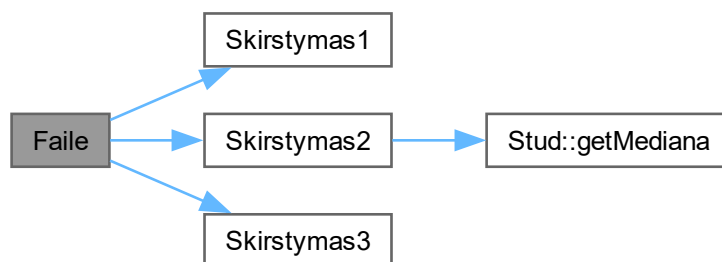
```
void Faile (  
    deque< Zmogus * > & grupe,  
    char gal,  
    double & TotalTime)
```

Funkcija studentų duomenų išvedimui į failą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.2.1.4 GeneruotiFaila()

```
void GeneruotiFaila (  
    double & TotalTime)
```

Funkcija studentų failo generavimui.

Parametrai

<i>TotalTime</i>	Bendras operacijos laikas.
------------------	----------------------------

Here is the caller graph for this function:



6.2.1.5 Manual()

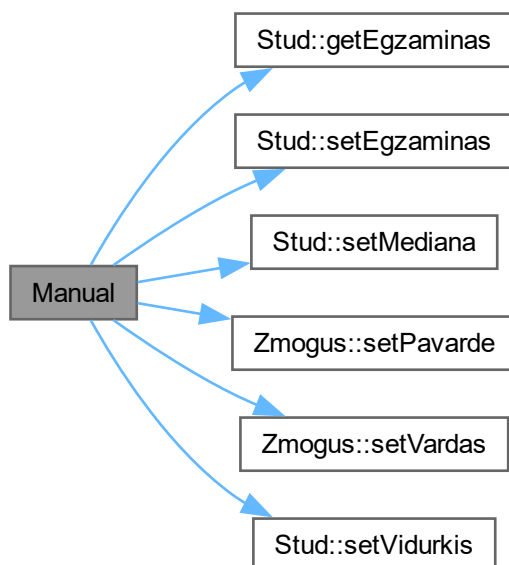
```
void Manual (  
    Stud & laik,  
    deque< Zmogus * > & grupe)
```

Funkcija studentų duomenų įvedimui rankiniu būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.2.1.6 Rusiuoti()

```

void Rusiuoti (
    deque< Zmogus * > & grupe,
    char rusiavimas,
    char gal,
    double & TotalTime)
  
```

Funkcija studentų rikiavimui pagal pasirinktą kriterijų.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>rusiavimas</i>	Rikiavimo kriterijus ('v' - vardas, 'p' - pavardė, 'g' - galutinis rezultatas).
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').

<i>TotalTime</i>	Bendras operacijos laikas.
------------------	----------------------------

Here is the caller graph for this function:



6.2.1.7 Semi()

```

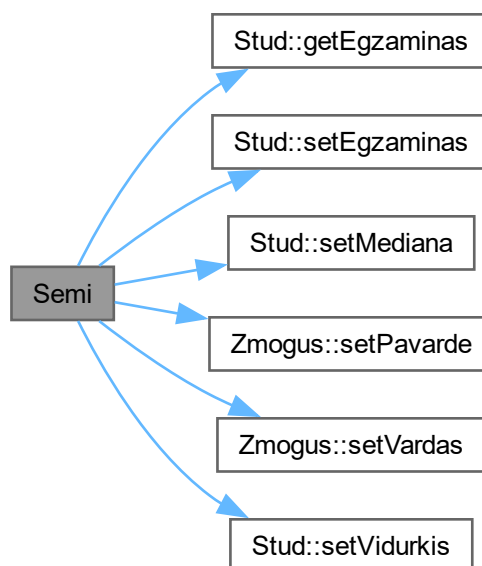
void Semi (
    Stud & laik,
    deque< Zmogus * > & grupe)
  
```

Funkcija studentų duomenų įvedimui pusiau automatinio būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.2.1.8 Skirstymas1()

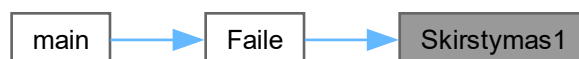
```
void Skirstymas1 (  
    deque< Zmogus * > & grupe,  
    char gal,  
    vector< Stud > & vargsiukai,  
    vector< Stud > & galvociai,  
    double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.2.1.9 Skirstymas2()

```
void Skirstymas2 (  
    deque< Zmogus * > & grupe,  
    char gal,  
    vector< Stud > & vargsiukai,  
    double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').

Parametrai

<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.2.1.10 Skirstymas3()

```

void Skirstymas3 (
    deque< Zmogus * > & grupe,
    char gal,
    vector< Stud > & vargsiukai,
    vector< Stud > & galvociai,
    double & TotalTime)
  
```

Funkcija studentų skirstymui į dvi grupes naudojant `std::partition`.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:

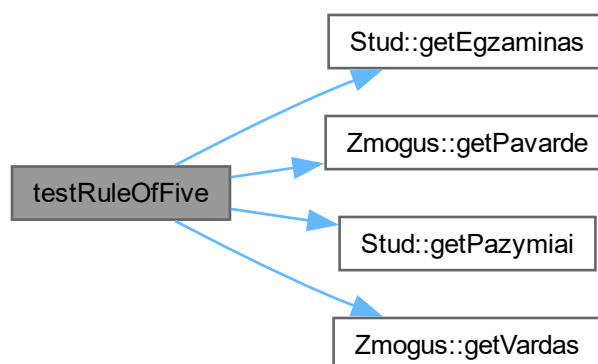


6.2.1.11 testRuleOfFive()

```
void testRuleOfFive ()
```

Funkcija Rule of Five taisyklės testavimui.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:

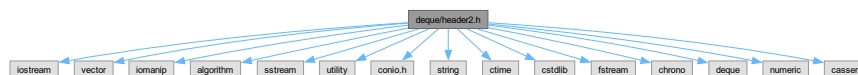


6.3 deque/header2.h Failo Nuoroda

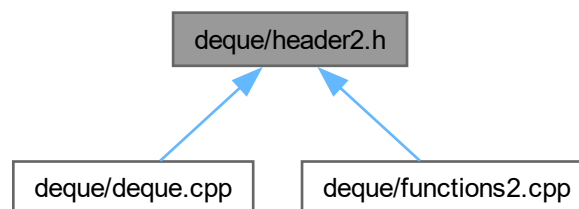
```
#include <iostream>
#include <vector>
#include <iomanip>
#include <algorithm>
```

```
#include <sstream>
#include <utility>
#include <conio.h>
#include <string>
#include <ctime>
#include <cstdlib>
#include <fstream>
#include <chrono>
#include <deque>
#include <numeric>
#include <cassert>
```

Įtraukimo priklausomybių diagrama header2.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class [Zmogus](#)
Bazinė/abstrakti klasė.
- struct [Stud](#)
Išvestinė klasė.

Funkcijos

- void [Manual](#) ([Stud](#) &laik, deque< [Zmogus](#) * > &grupe)
Funkcija studentų duomenų įvedimui rankiniu būdu.
- void [Semi](#) ([Stud](#) &laik, deque< [Zmogus](#) * > &grupe)
Funkcija studentų duomenų įvedimui pusiau automatinio būdu.
- void [Auto](#) (deque< [Zmogus](#) * > &grupe)
Funkcija studentų duomenų generavimui automatiškai.
- void [Ekranė](#) (deque< [Zmogus](#) * > &grupe, char gal, double &TotalTime)
Funkcija studentų duomenų išvedimui į ekraną.
- void [Skirstymas1](#) (deque< [Zmogus](#) * > &grupe, char gal, vector< [Stud](#) > &vargsiukai, vector< [Stud](#) > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.

- void [Skirstymas2](#) (deque< [Zmogus](#) * > &grupe, char gal, vector< [Stud](#) > &vargsiukai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.
- void [Skirstymas3](#) (deque< [Zmogus](#) * > &grupe, char gal, vector< [Stud](#) > &vargsiukai, vector< [Stud](#) > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes naudojant `std::partition`.
- void [Faile](#) (deque< [Zmogus](#) * > &grupe, char gal, double &TotalTime)
Funkcija studentų duomenų išvedimui į failą.
- void [Rusiuoti](#) (deque< [Zmogus](#) * > &grupe, char rusiavimas, char gal, double &TotalTime)
Funkcija studentų rikiavimui pagal pasirinktą kriterijų.
- void [GeneruotiFaila](#) (double &TotalTime)
Funkcija studentų failo generavimui.
- void [testRuleOfFive](#) ()
Funkcija Rule of Five taisyklės testavimui.

6.3.1 Funkcijos Dokumentacija

6.3.1.1 Auto()

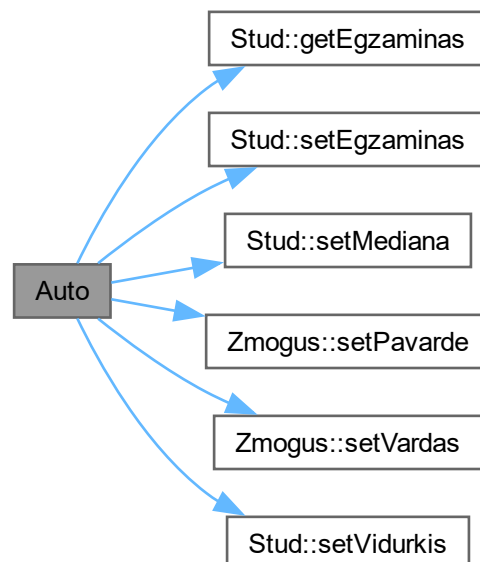
```
void Auto (
    deque< Zmogus * > & grupe)
```

Funkcija studentų duomenų generavimui automatiškai.

Parametrai

<i>grupe</i>	Studentų grupė.
--------------	-----------------

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.3.1.2 Ekrane()

```
void Ekrane (  
    deque< Zmogus * > & grupe,  
    char gal,  
    double & TotalTime)
```

Funkcija studentų duomenų išvedimui į ekraną.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.3.1.3 Faile()

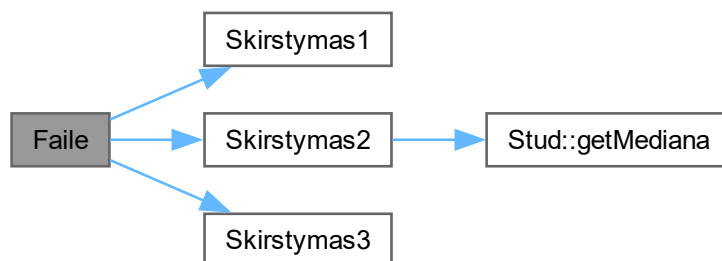
```
void Faile (  
    deque< Zmogus * > & grupe,  
    char gal,  
    double & TotalTime)
```

Funkcija studentų duomenų išvedimui į failą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.3.1.4 GeneruotiFaila()

```
void GeneruotiFaila (
    double & TotalTime)
```

Funkcija studentų failo generavimui.

Parametrai

<i>TotalTime</i>	Bendras operacijos laikas.
------------------	----------------------------

6.3.1.5 Manual()

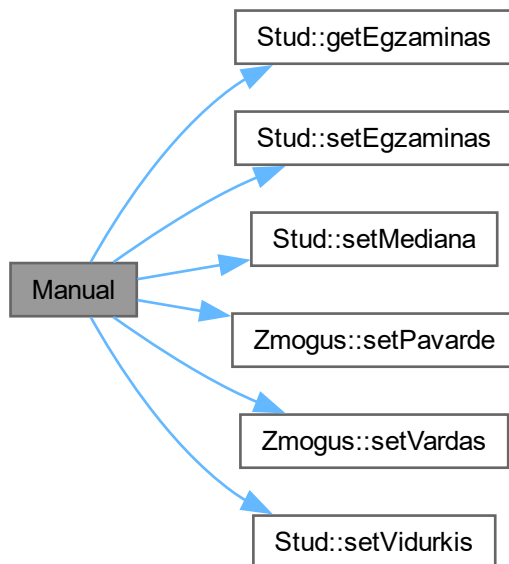
```
void Manual (
    Stud & laik,
    deque< Zmogus * > & grupe)
```

Funkcija studentų duomenų įvedimui rankiniu būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.3.1.6 Rusiuoti()

```

void Rusiuoti (
    deque< Zmogus * > & grupe,
    char rusiavimas,
    char gal,
    double & TotalTime)
  
```

Funkcija studentų rikiavimui pagal pasirinktą kriterijų.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>rusiavimas</i>	Rikiavimo kriterijus ('v' - vardas, 'p' - pavardė, 'g' - galutinis rezultatas).
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').

<i>TotalTime</i>	Bendras operacijos laikas.
------------------	----------------------------

Here is the caller graph for this function:



6.3.1.7 Semi()

```

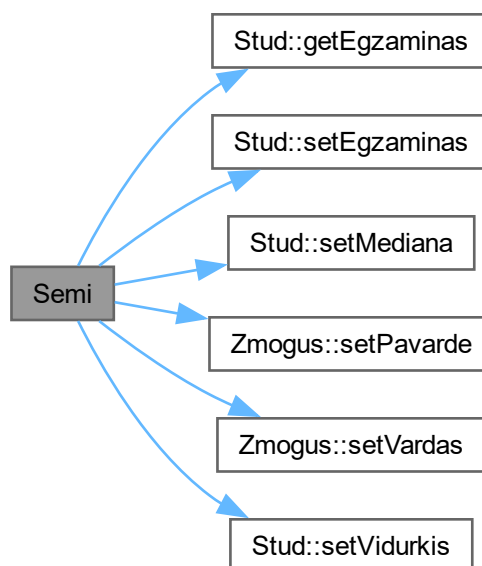
void Semi (
    Stud & laik,
    deque< Zmogus * > & grupe)
  
```

Funkcija studentų duomenų įvedimui pusiau automatinio būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.3.1.8 Skirstymas1()

```

void Skirstymas1 (
    deque< Zmogus * > & grupe,
    char gal,
    vector< Stud > & vargsiukai,
    vector< Stud > & galvociai,
    double & TotalTime)
  
```

Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.3.1.9 Skirstymas2()

```

void Skirstymas2 (
    deque< Zmogus * > & grupe,
    char gal,
    vector< Stud > & vargsiukai,
    double & TotalTime)
  
```

Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').

Parametrai

<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.3.1.10 Skirstymas3()

```

void Skirstymas3 (
    deque< Zmogus * > & grupe,
    char gal,
    vector< Stud > & vargsiukai,
    vector< Stud > & galvociai,
    double & TotalTime)
  
```

Funkcija studentų skirstymui į dvi grupes naudojant `std::partition`.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:

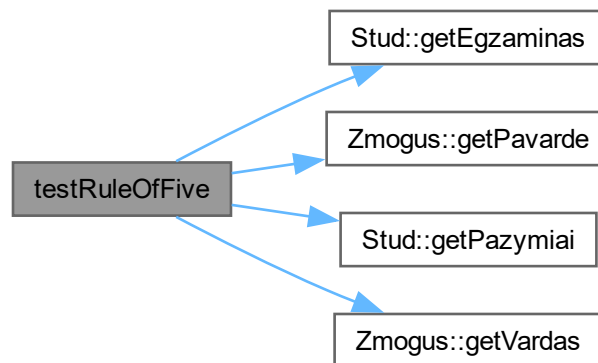


6.3.1.11 testRuleOfFive()

```
void testRuleOfFive ()
```

Funkcija Rule of Five taisyklės testavimui.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.4 header2.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #include <iostream>
00002 #include <vector>
00003 #include <iomanip>
00004 #include <algorithm>
```

```

00005 #include <sstream>
00006 #include <utility>
00007 #include <conio.h>
00008 #include <string>
00009 #include <ctime>
00010 #include <cstdlib>
00011 #include <fstream>
00012 #include <chrono>
00013 #include <deque>
00014 #include <numeric>
00015 #include <cassert>
00016 #pragma once
00017
00018 using std::cin;
00019 using std::cout;
00020 using std::deque;
00021 using std::endl;
00022 using std::fixed;
00023 using std::ifstream;
00024 using std::ofstream;
00025 using std::setprecision;
00026 using std::setw;
00027 using std::string;
00028 using std::to_string;
00029 using std::vector;
00030
00031 class Zmogus
00032 {
00033 protected:
00034     string vard, pav;
00035
00036 public:
00037     Zmogus() : vard(""), pav("") {}
00038
00039     Zmogus(const string &vardas, const string &pavarde) : vard(vardas), pav(pavarde) {}
00040
00041     virtual ~Zmogus() = default;
00042
00043     inline string getVardas() const { return vard; }
00044     inline string getPavarde() const { return pav; }
00045
00046     inline void setVardas(const string &vardas) { vard = vardas; }
00047     inline void setPavarde(const string &pavarde) { pav = pavarde; }
00048
00049     virtual void Skaityti(deque<Zmogus *> &grupe, double &TotalTime) = 0;
00050 };
00051
00052 class Stud : public Zmogus
00053 {
00054 private:
00055     vector<int> paz;
00056     int egz;
00057     double vid;
00058     double med;
00059
00060 public:
00061     Stud() : Zmogus(), egz(0), vid(0.0), med(0.0) {}
00062
00063     Stud(const string &vardas, const string &pavarde, const vector<int> &pazymiai, int egzaminas)
00064         : Zmogus(vardas, pavarde), paz(pazymiai), egz(egzaminas), vid(0.0), med(0.0) {}
00065
00066     ~Stud();
00067
00068     Stud(const Stud &other)
00069         : Zmogus(other.vard, other.pav), paz(other.paz), egz(other.egz), vid(other.vid),
00070         med(other.med) {}
00071
00072     Stud &operator=(const Stud &other)
00073     {
00074         if (this == &other)
00075             return *this;
00076         Zmogus::operator=(other);
00077         paz = other.paz;
00078         egz = other.egz;
00079         vid = other.vid;
00080         med = other.med;
00081         return *this;
00082     }
00083
00084     Stud(Stud &&other) noexcept
00085         : Zmogus(std::move(other.vard), std::move(other.pav)), paz(std::move(other.paz)),
00086         egz(other.egz), vid(other.vid), med(other.med)
00087     {
00088         other.vard.clear();
00089         other.pav.clear();
00090         other.egz = 0;
00091         other.vid = 0.0;
00092     }

```

```

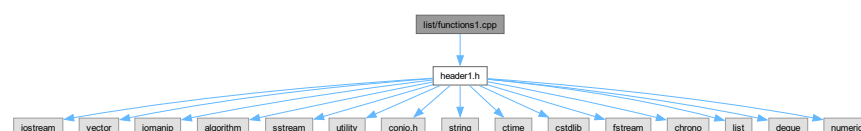
00151         other.med = 0.0;
00152     }
00153
00160     Stud &operator=(Stud &&other) noexcept
00161     {
00162         if (this == &other)
00163             return *this;
00164         Zmogus::operator=(std::move(other));
00165         paz = std::move(other.paz);
00166         egz = other.egz;
00167         vid = other.vid;
00168         med = other.med;
00169
00170         other.vard.clear();
00171         other.pav.clear();
00172         other.egz = 0;
00173         other.vid = 0.0;
00174         other.med = 0.0;
00175         return *this;
00176     }
00177
00181     inline vector<int> getPazymiai() const { return paz; }
00182     inline int getEgzaminas() const { return egz; }
00183     inline double getVidurkis() const { return vid; }
00184     inline double getMediana() const { return med; }
00185
00189     inline void setPazymiai(const vector<int> &pazymiai) { paz = pazymiai; }
00190     inline void setEgzaminas(int egzaminas) { egz = egzaminas; }
00191     inline void setVidurkis(double vidurkis) { vid = vidurkis; }
00192     inline void setMediana(double mediana) { med = mediana; }
00193
00200     void Skaityti(deque<Zmogus *> &grupe, double &TotalTime) override;
00201 };
00202
00209 void Manual(Stud &laik, deque<Zmogus *> &grupe);
00210
00217 void Semi(Stud &laik, deque<Zmogus *> &grupe);
00218
00224 void Auto(deque<Zmogus *> &grupe);
00225
00233 void Ekrane(deque<Zmogus *> &grupe, char gal, double &TotalTime);
00234
00244 void Skirstymas1(deque<Zmogus *> &grupe, char gal, vector<Stud> &vargsiukai, vector<Stud> &galvociai,
00245                 double &TotalTime);
00245
00254 void Skirstymas2(deque<Zmogus *> &grupe, char gal, vector<Stud> &vargsiukai, double &TotalTime);
00255
00265 void Skirstymas3(deque<Zmogus *> &grupe, char gal, vector<Stud> &vargsiukai, vector<Stud> &galvociai,
00266                 double &TotalTime);
00266
00274 void Faile(deque<Zmogus *> &grupe, char gal, double &TotalTime);
00275
00284 void Rusiuoti(deque<Zmogus *> &grupe, char rusiavimas, char gal, double &TotalTime);
00285
00291 void GeneruotiFaila(double &TotalTime);
00292
00296 void testRuleOffFive();

```

6.5 list/functions1.cpp Failo Nuoroda

#include "header1.h"

Įtraukimo priklausomybių diagrama functions1.cpp:



Funkcijos

- void **Manual** (Stud &laik, list< Stud > &grupe)
Funkcija studentų duomenų įvedimui rankiniu būdu.
- void **Semi** (Stud &laik, list< Stud > &grupe)

- Funkcija studentų duomenų įvedimui pusiau automatinio būdu.*
- void **Auto** (list< **Stud** > &grupe)
- Funkcija studentų duomenų generavimui automatiškai.*
- void **Skaityti** (**Stud** &laik, list< **Stud** > &grupe, double &TotalTime)
- Funkcija studentų duomenų skaitymui iš failo.*
- void **Ekrane** (list< **Stud** > &grupe, char gal, double &TotalTime)
- Funkcija studentų duomenų išvedimui į ekraną.*
- void **Skirstymas1** (list< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, vector< **Stud** > &galvociai, double &TotalTime)
- Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.*
- void **Skirstymas2** (list< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, double &TotalTime)
- Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.*
- void **Skirstymas3** (list< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, vector< **Stud** > &galvociai, double &TotalTime)
- Funkcija studentų skirstymui į dvi grupes naudojant std::partition.*
- void **Faile** (list< **Stud** > &grupe, char gal, double &TotalTime)
- Funkcija studentų duomenų išvedimui į failą.*
- void **Rusiuoti** (list< **Stud** > &grupe, char rusiavimas, char gal, double &TotalTime)
- Funkcija studentų rikiavimui pagal pasirinktą kriterijų.*
- void **GeneruotiFaila** (double &TotalTime)
- Funkcija studentų failo generavimui.*

6.5.1 Funkcijos Dokumentacija

6.5.1.1 Auto()

```
void Auto (
    list< Stud > & grupe)
```

Funkcija studentų duomenų generavimui automatiškai.

Parametrai

<i>grupe</i>	Studentų grupė.
--------------	-----------------

6.5.1.2 Ekrane()

```
void Ekrane (
    list< Stud > & grupe,
    char gal,
    double & TotalTime)
```

Funkcija studentų duomenų išvedimui į ekraną.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

6.5.1.3 Faile()

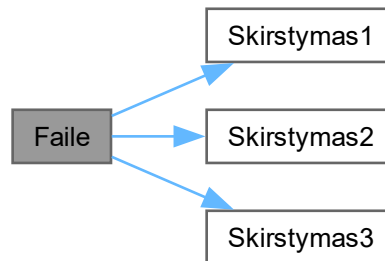
```
void Faile (
    list< Stud > & grupe,
    char gal,
    double & TotalTime)
```

Funkcija studentų duomenų išvedimui į failą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



6.5.1.4 GeneruotiFaila()

```
void GeneruotiFaila (
    double & TotalTime)
```

Funkcija studentų failo generavimui.

Parametrai

<i>TotalTime</i>	Bendras operacijos laikas.
------------------	----------------------------

6.5.1.5 Manual()

```
void Manual (
    Stud & laik,
    list< Stud > & grupe)
```

Funkcija studentų duomenų įvedimui rankiniu būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

6.5.1.6 Rusiuoti()

```
void Rusiuoti (
    list< Stud > & grupe,
    char rusiavimas,
    char gal,
    double & TotalTime)
```

Funkcija studentų rikiavimui pagal pasirinktą kriterijų.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>rusiavimas</i>	Rikiavimo kriterijus ('v' - vardas, 'p' - pavardė, 'g' - galutinis rezultatas).
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

6.5.1.7 Semi()

```
void Semi (
    Stud & laik,
    list< Stud > & grupe)
```

Funkcija studentų duomenų įvedimui pusiau automatinio būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

6.5.1.8 Skaityti()

```
void Skaityti (
    Stud & laik,
    list< Stud > & grupe,
    double & TotalTime)
```

Funkcija studentų duomenų skaitymui iš failo.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.5.1.9 Skirstymas1()

```
void Skirstymas1 (
    list< Stud > & grupe,
    char gal,
    vector< Stud > & vargsiukai,
    vector< Stud > & galvociiai,
    double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.5.1.10 Skirstymas2()

```

void Skirstymas2 (
    list< Stud > & grupe,
    char gal,
    vector< Stud > & vargsiukai,
    double & TotalTime)
  
```

Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.5.1.11 Skirstymas3()

```

void Skirstymas3 (
    list< Stud > & grupe,
    char gal,
    vector< Stud > & vargsiukai,
  
```

```
vector< Stud > & galvociai,
double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes naudojant `std::partition`.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



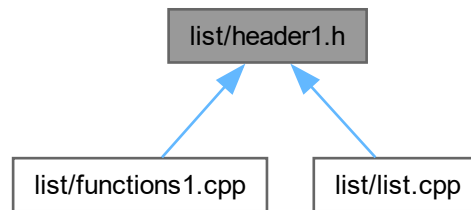
6.6 list/header1.h Failo Nuoroda

```
#include <iostream>
#include <vector>
#include <iomanip>
#include <algorithm>
#include <sstream>
#include <utility>
#include <conio.h>
#include <string>
#include <ctime>
#include <cstdlib>
#include <fstream>
#include <chrono>
#include <list>
#include <deque>
#include <numeric>
```

Įtraukimo priklausomybių diagrama header1.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- struct **Stud**
Išvestinė klasė.

Funkcijos

- void **Manual** (**Stud** &laik, list< **Stud** > &grupe)
Funkcija studentų duomenų įvedimui rankiniu būdu.
- void **Semi** (**Stud** &laik, list< **Stud** > &grupe)
Funkcija studentų duomenų įvedimui pusiau automatinio būdu.
- void **Auto** (list< **Stud** > &grupe)
Funkcija studentų duomenų generavimui automatiškai.
- void **Skaityti** (**Stud** &laik, list< **Stud** > &grupe, double &TotalTime)
Funkcija studentų duomenų skaitymui iš failo.
- void **Ekrane** (list< **Stud** > &grupe, char gal, double &TotalTime)
Funkcija studentų duomenų išvedimui į ekraną.
- void **Skirstymas1** (list< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, vector< **Stud** > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.
- void **Skirstymas2** (list< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.
- void **Skirstymas3** (list< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, vector< **Stud** > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes naudojant std::partition.
- void **Faile** (list< **Stud** > &grupe, char gal, double &TotalTime)
Funkcija studentų duomenų išvedimui į failą.
- void **Rusiuoti** (list< **Stud** > &grupe, char rusiavimas, char gal, double &TotalTime)
Funkcija studentų rikiavimui pagal pasirinktą kriterijų.
- void **GeneruotiFaila** (double &TotalTime)
Funkcija studentų failo generavimui.

6.6.1 Funkcijos Dokumentacija

6.6.1.1 Auto()

```
void Auto (
    list< Stud > &grupe)
```

Funkcija studentų duomenų generavimui automatiškai.

Parametrai

<i>grupe</i>	Studentų grupė.
--------------	-----------------

6.6.1.2 Ekrane()

```
void Ekrane (
    list< Stud > & grupe,
    char gal,
    double & TotalTime)
```

Funkcija studentų duomenų išvedimui į ekraną.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

6.6.1.3 Faile()

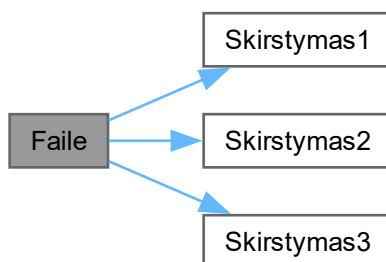
```
void Faile (
    list< Stud > & grupe,
    char gal,
    double & TotalTime)
```

Funkcija studentų duomenų išvedimui į failą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



6.6.1.4 GeneruotiFaila()

```
void GeneruotiFaila (  
    double & TotalTime)
```

Funkcija studentų failo generavimui.

Parametrai

<i>TotalTime</i>	Bendras operacijos laikas.
------------------	----------------------------

6.6.1.5 Manual()

```
void Manual (
    Stud & laik,
    list< Stud > & grupe)
```

Funkcija studentų duomenų įvedimui rankiniu būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

6.6.1.6 Rusiuoti()

```
void Rusiuoti (
    list< Stud > & grupe,
    char rusiavimas,
    char gal,
    double & TotalTime)
```

Funkcija studentų rikiavimui pagal pasirinktą kriterijų.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>rusiavimas</i>	Rikiavimo kriterijus ('v' - vardas, 'p' - pavardė, 'g' - galutinis rezultatas).
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

6.6.1.7 Semi()

```
void Semi (
    Stud & laik,
    list< Stud > & grupe)
```

Funkcija studentų duomenų įvedimui pusiau automatinio būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

6.6.1.8 Skaityti()

```
void Skaityti (
    Stud & laik,
    list< Stud > & grupe,
    double & TotalTime)
```

Funkcija studentų duomenų skaitymui iš failo.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.6.1.9 Skirstymas1()

```

void Skirstymas1 (
    list< Stud > & grupe,
    char gal,
    vector< Stud > & vargsiukai,
    vector< Stud > & galvociai,
    double & TotalTime)
  
```

Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.6.1.10 Skirstymas2()

```

void Skirstymas2 (
    list< Stud > & grupe,
    char gal,
  
```

```
vector< Stud > & vargsiukai,
double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.6.1.11 Skirstymas3()

```
void Skirstymas3 (
    list< Stud > & grupe,
    char gal,
    vector< Stud > & vargsiukai,
    vector< Stud > & galvociai,
    double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes naudojant `std::partition`.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.7 header1.h

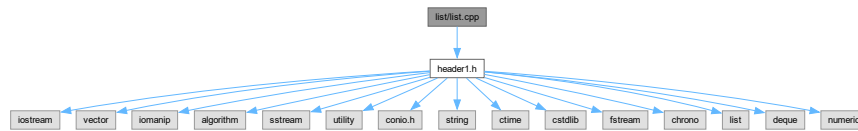
[Eiti į šio failo dokumentaciją.](#)

```
00001 #include <iostream>
00002 #include <vector>
00003 #include <iomanip>
00004 #include <algorithm>
00005 #include <sstream>
00006 #include <utility>
00007 #include <conio.h>
00008 #include <string>
00009 #include <ctime>
00010 #include <cstdlib>
00011 #include <fstream>
00012 #include <chrono>
00013 #include <list>
00014 #include <deque>
00015 #include <algorithm>
00016 #include <numeric>
00017 #pragma once
00018
00019 using std::cout;
00020 using std::cin;
00021 using std::endl;
00022 using std::string;
00023 using std::vector;
00024 using std::fixed;
00025 using std::setprecision;
00026 using std::setw;
00027 using std::ifstream;
00028 using std::ofstream;
00029 using std::to_string;
00030 using std::list;
00031 using std::sort;
00032
00036 struct Stud
00037 {
00038     string vard;
00039     string pav;
00040     vector<int> paz;
00041     int egz;
00042     double vid;
00043     double med;
00044 };
00045
00052 void Manual(Stud &laik, list<Stud> &grupe);
00053
00060 void Semi(Stud &laik, list<Stud> &grupe);
00061
00067 void Auto(list<Stud> &grupe);
00068
00076 void Skaityti(Stud &laik, list<Stud> &grupe, double &TotalTime);
00077
00085 void Ekране(list<Stud> &grupe, char gal, double &TotalTime);
00086
00096 void Skirstymas1(list<Stud> &grupe, char gal, vector<Stud> &vargsiukai, vector<Stud> &galvociai,
    double &TotalTime);
00097
00106 void Skirstymas2(list<Stud> &grupe, char gal, vector<Stud> &vargsiukai, double &TotalTime);
00107
00117 void Skirstymas3(list<Stud> &grupe, char gal, vector<Stud> &vargsiukai, vector<Stud> &galvociai,
    double &TotalTime);
00118
00126 void Faile(list<Stud> &grupe, char gal, double &TotalTime);
00127
00136 void Rusiuoti(list<Stud> &grupe, char rusiavimas, char gal, double &TotalTime);
00137
00143 void GeneruotiFaila(double &TotalTime);
```

6.8 list/list.cpp Failo Nuoroda

```
#include "header1.h"
```

Įtraukimo priklausomybių diagrama list.cpp:



Funkcijos

- int `main` ()

Pagrindinė programos funkcija.

6.8.1 Funkcijos Dokumentacija

6.8.1.1 main()

```
int main ()
```

Pagrindinė programos funkcija.

Ši funkcija leidžia vartotojui pasirinkti įvairias operacijas, tokias kaip studentų duomenų generavimas, įvedimas, rikiavimas, skirstymas ir išvedimas į ekraną arba failą.

Gražina

int Programos vykdymo rezultatas.

Tikrinama, ar vartotojas nori sugeneruoti studentų failą.

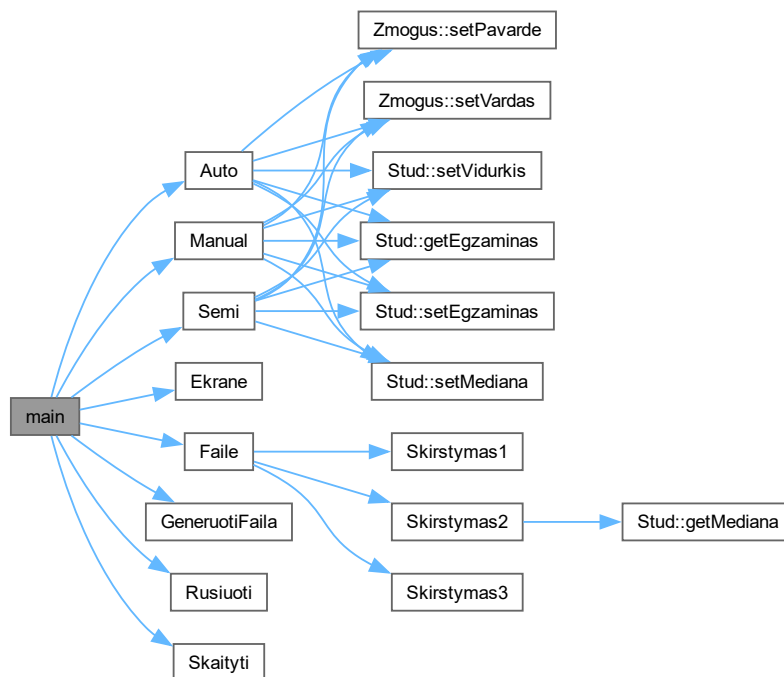
Leidžiama pasirinkti studentų duomenų įvedimo būdą.

Pasirenkama, ar naudoti vidurkį, ar medianą.

Pasirenkamas rikiavimo kriterijus.

Pasirenkama, ar duomenis išvesti į ekraną, ar į failą.

Išvedamas bendras programos veikimo laikas. Funkcijos kvietimo grafas:

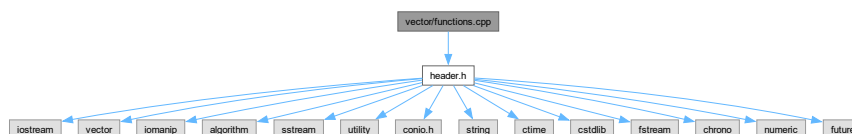


6.9 README.md Failo Nuoroda

6.10 vector/functions.cpp Failo Nuoroda

```
#include "header.h"
```

Įtraukimo priklausomybių diagrama functions.cpp:



Funkcijos

- void **Manual** (Stud &laik, vector< Stud > &grupe)
Funkcija studentų duomenų įvedimui rankiniu būdu.
- void **Semi** (Stud &laik, vector< Stud > &grupe)
Funkcija studentų duomenų įvedimui pusiau automatinio būdu.
- void **Auto** (vector< Stud > &grupe)
Funkcija studentų duomenų generavimui automatiškai.
- void **Skaityti** (Stud &laik, vector< Stud > &grupe, double &TotalTime)
Funkcija studentų duomenų skaitymui iš failo.
- void **Ekrane** (vector< Stud > &grupe, char gal, double &TotalTime)

Funkcija studentų duomenų išvedimui į ekraną.

- void **Skirstymas1** (vector< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, vector< **Stud** > &galvociai, double &TotalTime)

Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.

- void **Skirstymas2** (vector< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, double &TotalTime)

Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.

- void **Skirstymas3** (vector< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, vector< **Stud** > &galvociai, double &TotalTime)
- void **Faile** (vector< **Stud** > &grupe, char gal, double &TotalTime)

Funkcija studentų duomenų išvedimui į failą.

- void **Rusiuoti** (vector< **Stud** > &grupe, char rusiavimas, char gal, double &TotalTime)

Funkcija studentų rikiavimui pagal pasirinktą kriterijų.

- void **GeneruotiFaila** (double &TotalTime)

Funkcija studentų failo generavimui.

6.10.1 Funkcijos Dokumentacija

6.10.1.1 Auto()

```
void Auto (
    vector< Stud > &grupe)
```

Funkcija studentų duomenų generavimui automatiškai.

Parametrai

<i>grupe</i>	Studentų grupė.
--------------	-----------------

6.10.1.2 Ekrane()

```
void Ekrane (
    vector< Stud > &grupe,
    char gal,
    double &TotalTime)
```

Funkcija studentų duomenų išvedimui į ekraną.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

6.10.1.3 Faile()

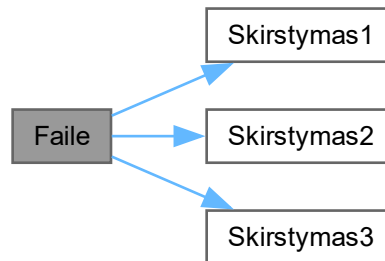
```
void Faile (
    vector< Stud > &grupe,
    char gal,
    double &TotalTime)
```

Funkcija studentų duomenų išvedimui į failą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



6.10.1.4 GeneruotiFaila()

```
void GeneruotiFaila (
    double & TotalTime)
```

Funkcija studentų failo generavimui.

Parametrai

<i>TotalTime</i>	Bendras operacijos laikas.
------------------	----------------------------

6.10.1.5 Manual()

```
void Manual (
    Stud & laik,
    vector< Stud > & grupe)
```

Funkcija studentų duomenų įvedimui rankiniu būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

6.10.1.6 Rusiuoti()

```
void Rusiuoti (
    vector< Stud > & grupe,
    char rusiavimas,
    char gal,
    double & TotalTime)
```

Funkcija studentų rikiavimui pagal pasirinktą kriterijų.

Parametrai

<i>grupe</i>	Studentų grupė.
--------------	-----------------

Parametrai

<i>rusiavimas</i>	Rikiavimo kriterijus ('v' - vardas, 'p' - pavardė, 'g' - galutinis rezultatas).
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

6.10.1.7 Semi()

```
void Semi (
    Stud & laik,
    vector< Stud > & grupe)
```

Funkcija studentų duomenų įvedimui pusiau automatinio būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

6.10.1.8 Skaityti()

```
void Skaityti (
    Stud & laik,
    vector< Stud > & grupe,
    double & TotalTime)
```

Funkcija studentų duomenų skaitymui iš failo.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.
<i>TotalTime</i>	Bendras operacijos laikas.

6.10.1.9 Skirstymas1()

```
void Skirstymas1 (
    vector< Stud > & grupe,
    char gal,
    vector< Stud > & vargsiukai,
    vector< Stud > & galvociiai,
    double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociiai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.10.1.10 Skirstymas2()

```
void Skirstymas2 (  
    vector< Stud > & grupe,  
    char gal,  
    vector< Stud > & vargsiukai,  
    double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>TotalTime</i>	Bendras operacijos laikas.

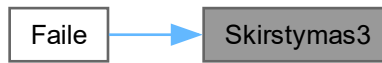
Here is the caller graph for this function:



6.10.1.11 Skirstymas3()

```
void Skirstymas3 (  
    vector< Stud > & grupe,  
    char gal,  
    vector< Stud > & vargsiukai,  
    vector< Stud > & galvociai,  
    double & TotalTime)
```

Here is the caller graph for this function:

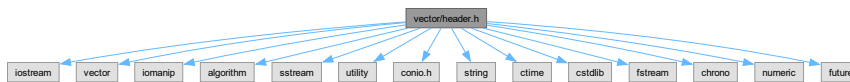


6.11 vector/header.h Failo Nuoroda

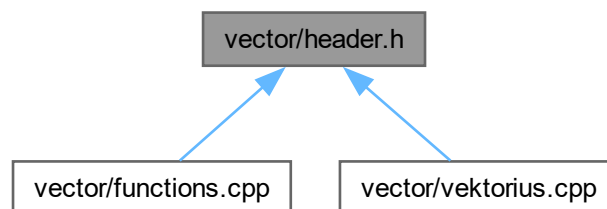
```

#include <iostream>
#include <vector>
#include <iomanip>
#include <algorithm>
#include <sstream>
#include <utility>
#include <conio.h>
#include <string>
#include <ctime>
#include <cstdlib>
#include <fstream>
#include <chrono>
#include <numeric>
#include <future>
  
```

Įtraukimo priklausomybių diagrama header.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- struct [Stud](#)
Išvestinė klasė.

Funkcijos

- void **Manual** (**Stud** &laik, vector< **Stud** > &grupe)
Funkcija studentų duomenų įvedimui rankiniu būdu.
- void **Semi** (**Stud** &laik, vector< **Stud** > &grupe)
Funkcija studentų duomenų įvedimui pusiau automatinio būdu.
- void **Auto** (vector< **Stud** > &grupe)
Funkcija studentų duomenų generavimui automatiškai.
- void **Skaityti** (**Stud** &laik, vector< **Stud** > &grupe, double &TotalTime)
Funkcija studentų duomenų skaitymui iš failo.
- void **Ekrane** (vector< **Stud** > &grupe, char gal, double &TotalTime)
Funkcija studentų duomenų išvedimui į ekraną.
- void **Skirstymas1** (vector< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, vector< **Stud** > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.
- void **Skirstymas2** (vector< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.
- void **skirstymas3** (vector< **Stud** > &grupe, char gal, vector< **Stud** > &vargsiukai, vector< **Stud** > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes naudojant `std::partition`.
- void **Faila** (vector< **Stud** > &grupe, char gal, double &TotalTime)
Funkcija studentų duomenų išvedimui į failą.
- void **Rusiuoti** (vector< **Stud** > &grupe, char rusiavimas, char gal, double &TotalTime)
Funkcija studentų rikiavimui pagal pasirinktą kriterijų.
- void **GeneruotiFaila** (double &TotalTime)
Funkcija studentų failo generavimui.

6.11.1 Funkcijos Dokumentacija

6.11.1.1 Auto()

```
void Auto (
    vector< Stud > & grupe)
```

Funkcija studentų duomenų generavimui automatiškai.

Parametrai

<i>grupe</i>	Studentų grupė.
--------------	-----------------

6.11.1.2 Ekrane()

```
void Ekrane (
    vector< Stud > & grupe,
    char gal,
    double & TotalTime)
```

Funkcija studentų duomenų išvedimui į ekraną.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

6.11.1.3 Faile()

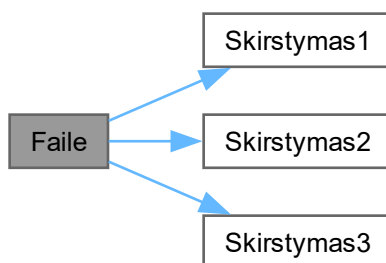
```
void Faile (  
    vector< Stud > & grupe,  
    char gal,  
    double & TotalTime)
```

Funkcija studentų duomenų išvedimui į failą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



6.11.1.4 GeneruotiFaila()

```
void GeneruotiFaila (  
    double & TotalTime)
```

Funkcija studentų failo generavimui.

Parametrai

<i>TotalTime</i>	Bendras operacijos laikas.
------------------	----------------------------

Here is the caller graph for this function:



6.11.1.5 Manual()

```
void Manual (  
    Stud & laik,  
    vector< Stud > & grupe)
```

Funkcija studentų duomenų įvedimui rankiniu būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

6.11.1.6 Rusiuoti()

```
void Rusiuoti (  
    vector< Stud > & grupe,  
    char rusiavimas,  
    char gal,  
    double & TotalTime)
```

Funkcija studentų rikiavimui pagal pasirinktą kriterijų.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>rusiavimas</i>	Rikiavimo kriterijus ('v' - vardas, 'p' - pavardė, 'g' - galutinis rezultatas).
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

6.11.1.7 Semi()

```
void Semi (  
    Stud & laik,  
    vector< Stud > & grupe)
```

Funkcija studentų duomenų įvedimui pusiau automatinio būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

6.11.1.8 Skaityti()

```
void Skaityti (  
    Stud & laik,  
    vector< Stud > & grupe,  
    double & TotalTime)
```

Funkcija studentų duomenų skaitymui iš failo.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.
<i>TotalTime</i>	Bendras operacijos laikas.

6.11.1.9 Skirstymas1()

```
void Skirstymas1 (  
    vector< Stud > & grupe,  
    char gal,  
    vector< Stud > & vargsiukai,  
    vector< Stud > & galvociiai,  
    double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociiai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.11.1.10 Skirstymas2()

```
void Skirstymas2 (  
    vector< Stud > & grupe,  
    char gal,  
    vector< Stud > & vargsiukai,  
    double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.11.1.11 skirstymas3()

```
void skirstymas3 (
    vector< Stud > & grupe,
    char gal,
    vector< Stud > & vargsiukai,
    vector< Stud > & galvociai,
    double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes naudojant `std::partition`.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

6.12 header.h

Eiti į šio failo dokumentaciją.

```
00001 #include <iostream>
00002 #include <vector>
00003 #include <iomanip>
00004 #include <algorithm>
00005 #include <sstream>
00006 #include <utility>
00007 #include <conio.h>
00008 #include <string>
00009 #include <ctime>
00010 #include <cstdlib>
00011 #include <fstream>
00012 #include <chrono>
00013 #include <numeric>
00014 #include <future>
00015 #pragma once
00016
00017 using std::cout;
00018 using std::cin;
00019 using std::endl;
00020 using std::string;
00021 using std::vector;
00022 using std::fixed;
00023 using std::setprecision;
00024 using std::setw;
00025 using std::ifstream;
00026 using std::ofstream;
00027 using std::to_string;
00028
00032 struct Stud
00033 {
00034     string vard;
00035     string pav;
00036     vector<int> paz;
00037     int egz;
00038     double vid;
00039     double med;
00040 };
00041
00048 void Manual(Stud &laik, vector<Stud> &grupe);
00049
00056 void Semi(Stud &laik, vector<Stud> &grupe);
00057
00063 void Auto(vector<Stud> &grupe);
00064
00072 void Skaityti(Stud &laik, vector<Stud> &grupe, double &TotalTime);
00073
00081 void Ekране(vector<Stud> &grupe, char gal, double &TotalTime);
00082
00092 void Skirstymas1(vector<Stud> &grupe, char gal, vector<Stud> &vargsiukai, vector<Stud> &galvociai,
    double &TotalTime);
00093
00102 void Skirstymas2(vector<Stud> &grupe, char gal, vector<Stud> &vargsiukai, double &TotalTime);
```

```

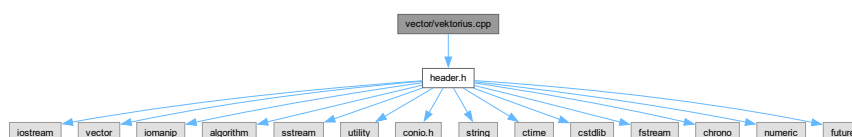
00103
00113 void skirstymas3(vector<Stud> &grupe, char gal, vector<Stud> &vargsiukai, vector<Stud> &galvociiai,
      double &TotalTime);
00114
00122 void Faile(vector<Stud> &grupe, char gal, double &TotalTime);
00123
00132 void Rusiuoti(vector<Stud> &grupe, char rusiavimas, char gal, double &TotalTime);
00133
00139 void GeneruotiFaila(double &TotalTime);

```

6.13 vector/vektorius.cpp Failo Nuoroda

```
#include "header.h"
```

Įtraukimo priklausomybių diagrama vektorius.cpp:



Funkcijos

- int main ()

Pagrindinė programos funkcija.

6.13.1 Funkcijos Dokumentacija

6.13.1.1 main()

```
int main ()
```

Pagrindinė programos funkcija.

Ši funkcija leidžia vartotojui pasirinkti įvairias operacijas, tokias kaip studentų duomenų generavimas, įvedimas, rikiavimas, skirstymas ir išvedimas į ekraną arba failą.

Gražina

int Programos vykdymo rezultatas.

Tikrinama, ar vartotojas nori sugeneruoti studentų failą.

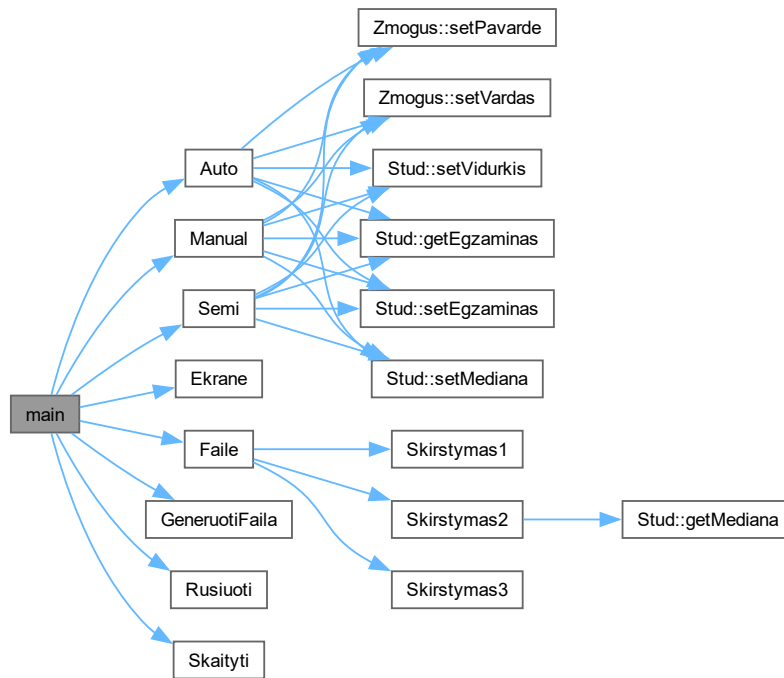
Leidžiama pasirinkti studentų duomenų įvedimo būdą.

Pasirenkama, ar naudoti vidurkį, ar medianą.

Pasirenkamas rikiavimo kriterijus.

Pasirenkama, ar duomenis išvesti į ekraną, ar į failą.

Išvedamas bendras programos veikimo laikas. Funkcijos kvietimo grafas:



Rodyklė

~Stud
Stud, [14](#)
~Zmogus
Zmogus, [22](#)

Auto
functions.cpp, [63](#)
functions1.cpp, [49](#)
functions2.cpp, [29](#)
header.h, [68](#)
header1.h, [54](#)
header2.h, [39](#)

deque.cpp
main, [27](#)
deque/deque.cpp, [27](#)
deque/functions2.cpp, [28](#)
deque/header2.h, [37](#), [46](#)

egz
Stud, [20](#)

Ekrane
functions.cpp, [63](#)
functions1.cpp, [49](#)
functions2.cpp, [30](#)
header.h, [68](#)
header1.h, [55](#)
header2.h, [40](#)

Faile
functions.cpp, [63](#)
functions1.cpp, [49](#)
functions2.cpp, [31](#)
header.h, [68](#)
header1.h, [55](#)
header2.h, [40](#)

functions.cpp
Auto, [63](#)
Ekrane, [63](#)
Faile, [63](#)
GeneruotiFaila, [64](#)
Manual, [64](#)
Rusiuoti, [64](#)
Semi, [65](#)
Skaityti, [65](#)
Skirstymas1, [65](#)
Skirstymas2, [66](#)
Skirstymas3, [66](#)

functions1.cpp
Auto, [49](#)

Ekrane, [49](#)
Faile, [49](#)
GeneruotiFaila, [50](#)
Manual, [50](#)
Rusiuoti, [50](#)
Semi, [51](#)
Skaityti, [51](#)
Skirstymas1, [51](#)
Skirstymas2, [52](#)
Skirstymas3, [52](#)
functions2.cpp
Auto, [29](#)
Ekrane, [30](#)
Faile, [31](#)
GeneruotiFaila, [32](#)
Manual, [32](#)
Rusiuoti, [33](#)
Semi, [34](#)
Skirstymas1, [35](#)
Skirstymas2, [35](#)
Skirstymas3, [36](#)
testRuleOfFive, [37](#)

GeneruotiFaila
functions.cpp, [64](#)
functions1.cpp, [50](#)
functions2.cpp, [32](#)
header.h, [69](#)
header1.h, [55](#)
header2.h, [41](#)

getEgzaminas
Stud, [16](#)

getMediana
Stud, [16](#)

getPavarde
Zmogus, [23](#)

getPazymiai
Stud, [16](#)

getVardas
Zmogus, [23](#)

getVidurkis
Stud, [17](#)

header.h
Auto, [68](#)
Ekrane, [68](#)
Faile, [68](#)
GeneruotiFaila, [69](#)
Manual, [69](#)
Rusiuoti, [70](#)

- Semi, [70](#)
- Skaityti, [70](#)
- Skirstymas1, [70](#)
- Skirstymas2, [71](#)
- skirstymas3, [72](#)
- header1.h
 - Auto, [54](#)
 - Ekrane, [55](#)
 - Faile, [55](#)
 - GeneruotiFaila, [55](#)
 - Manual, [57](#)
 - Rusiuoti, [57](#)
 - Semi, [57](#)
 - Skaityti, [57](#)
 - Skirstymas1, [58](#)
 - Skirstymas2, [58](#)
 - Skirstymas3, [59](#)
- header2.h
 - Auto, [39](#)
 - Ekrane, [40](#)
 - Faile, [40](#)
 - GeneruotiFaila, [41](#)
 - Manual, [41](#)
 - Rusiuoti, [42](#)
 - Semi, [43](#)
 - Skirstymas1, [44](#)
 - Skirstymas2, [44](#)
 - Skirstymas3, [45](#)
 - testRuleOfFive, [46](#)
- list.cpp
 - main, [61](#)
- list/functions1.cpp, [48](#)
- list/header1.h, [53](#), [60](#)
- list/list.cpp, [61](#)
- main
 - deque.cpp, [27](#)
 - list.cpp, [61](#)
 - vektorius.cpp, [73](#)
- Manual
 - functions.cpp, [64](#)
 - functions1.cpp, [50](#)
 - functions2.cpp, [32](#)
 - header.h, [69](#)
 - header1.h, [57](#)
 - header2.h, [41](#)
- med
 - Stud, [20](#)
- operator=
 - Stud, [17](#)
- pav
 - Stud, [20](#)
 - Zmogus, [25](#)
- paz
 - Stud, [21](#)
- Programos versijos, [1](#)
- README.md, [62](#)
- Rusiuoti
 - functions.cpp, [64](#)
 - functions1.cpp, [50](#)
 - functions2.cpp, [33](#)
 - header.h, [70](#)
 - header1.h, [57](#)
 - header2.h, [42](#)
- Semi
 - functions.cpp, [65](#)
 - functions1.cpp, [51](#)
 - functions2.cpp, [34](#)
 - header.h, [70](#)
 - header1.h, [57](#)
 - header2.h, [43](#)
- setEgzaminas
 - Stud, [18](#)
- setMediana
 - Stud, [18](#)
- setPavarde
 - Zmogus, [23](#)
- setPazymiai
 - Stud, [19](#)
- setVardas
 - Zmogus, [24](#)
- setVidurkis
 - Stud, [19](#)
- Skaityti
 - functions.cpp, [65](#)
 - functions1.cpp, [51](#)
 - header.h, [70](#)
 - header1.h, [57](#)
 - Stud, [19](#)
 - Zmogus, [24](#)
- Skirstymas1
 - functions.cpp, [65](#)
 - functions1.cpp, [51](#)
 - functions2.cpp, [35](#)
 - header.h, [70](#)
 - header1.h, [58](#)
 - header2.h, [44](#)
- Skirstymas2
 - functions.cpp, [66](#)
 - functions1.cpp, [52](#)
 - functions2.cpp, [35](#)
 - header.h, [71](#)
 - header1.h, [58](#)
 - header2.h, [44](#)
- Skirstymas3
 - functions.cpp, [66](#)
 - functions1.cpp, [52](#)
 - functions2.cpp, [36](#)
 - header1.h, [59](#)
 - header2.h, [45](#)
- skirstymas3
 - header.h, [72](#)
- Stud, [11](#)
 - ~Stud, [14](#)

- egz, [20](#)
- getEgzaminas, [16](#)
- getMediana, [16](#)
- getPazymiai, [16](#)
- getVidurkis, [17](#)
- med, [20](#)
- operator=, [17](#)
- pav, [20](#)
- paz, [21](#)
- setEgzaminas, [18](#)
- setMediana, [18](#)
- setPazymiai, [19](#)
- setVidurkis, [19](#)
- Skaityti, [19](#)
- Stud, [13](#), [14](#)
- vard, [21](#)
- vid, [21](#)

- testRuleOfFive
 - functions2.cpp, [37](#)
 - header2.h, [46](#)

- vard
 - Stud, [21](#)
 - Zmogus, [25](#)
- vector/functions.cpp, [62](#)
- vector/header.h, [67](#), [72](#)
- vector/vektorius.cpp, [73](#)
- vektorius.cpp
 - main, [73](#)
- vid
 - Stud, [21](#)

- Zmogus, [21](#)
 - ~Zmogus, [22](#)
 - getPavarde, [23](#)
 - getVardas, [23](#)
 - pav, [25](#)
 - setPavarde, [23](#)
 - setVardas, [24](#)
 - Skaityti, [24](#)
 - vard, [25](#)
 - Zmogus, [22](#)